

****

19年目フルスタックエンジニア(兼エンジニアリングマネージャー)

Contact

Website https://**********/profile/

Email **********

Phone **********

Location

City Minato-ku

Region 東京都

About

Im a full-stack engineer. Ive worked mostly at startups over than 46 projects.

Professional Skills Summary

Backend Development (19+ years): TypeScript, Go, Python, Rust, Clean Architecture, DDD, RESTful API, Swagger, OpenAPI, Serverless, GraphQL, Docker, docker-compose

Database (19+ years): MySQL, Postgres, DynamoDB, MongoDB, Redis, ElasticSearch, OpenSearch

DevOps (17+ years): CI/CD, GitHub Actions, Testing Automation, Documentation Automation, SchemaSpy, CloudMapper, E2E Testing, Unit Testing, Integration Testing, CircleCI

AI Integration (2+ years): OpenAI, Claude, Claude Code, Gemini, langchain, AutoGen, AutoGPT, Autonomous AI, AI Agent Orchestration

Infrastructure (12+ years): CDK, SST, Docker, docker-compose, AWS ECS Fargate, AWS Lambda, Terraform, Ansible, CloudFormation, Kubernetes

Frontend Development (15+ years): React, TypeScript, shadcn, Tailwind CSS, Storybook, BrowserStack, Vue, Angular, Aurelia

Management (4+ years): Agile, Lean, Trunk-Based Development, Full-Remote team leading, Full-Flexible team leading, git-flow, GitHub Flow, GitHub Issues, GitHub Projects

Dislike: Ruby, PHP, iOS, Scrum, JIRA, GitLab, Slack, Old, legacy environments

Work Experience

2025-12-14 to 現在

フルスタックエンジニア at OSSプロジェクト npm-cli-gh-issue-preparator

TypeScript, Clean Architecture, DDD, GitHub Actions, GitHub Issues, GitHub Projects, Jest, Node.js

- GitHub Issueの準備・自動化を行うCLIツールをOSSとして開発しました。
- 設定値の3階層解決（config YAML/CLIオプション/GitHub Project README）を実装し、柔軟な設定管理を実現しました。
- ワークフローブロッカー検知とWebhook通知機能を追加しました。
- CI未実行ジョブと失敗ジョブの区別検知機能を実装しました。
- GraphQL APIのエラーハンドリングを修正し、組織/ユーザーコンテキストのクエリに対応しました。
- issueに紐づく既存PRのブランチ名をGraphQL APIで自動取得し、awコマンドに--branchオプションで渡すことで、再トリガー時に既存ブランチを正しく引き継ぐ機能を実装しました。
- プロジェクトアイテムがPR URLの場合にfindRelatedOpenPRsが例外をスローする問題を修正しました。IssueRepositoryインターフェースにgetOpenPullRequest(prUrl)を追加し、collectRejections()でPR URLを検出して適切に処理するよう改修しました。
- issueにNextActionDateまたはNextActionHourが設定されている場合にAwaiting Quality CheckではなくAwaiting Workspaceへ自動移動するよう改善

し、依存issueありの場合と動作を統一しました。

- llm-agentラベル付きissueでNotifyFinishがPRの必須チェックを誤って要求する不具合を修正し、categoryラベルと同様のスキップ動作に統一しました。
- CLIをprogram.parse()からparseAsync()に移行してasync処理が確実に完了するようにし、デフォルトClaudeモデルをclaude-sonnet-4.6に設定しました。
- HiromiShikata配下の全管理対象リポジトリで廃止予定のcategory:*ラベルをgithub-label-syncのエイリアス機能を活用してダウンタイムなしでllm-agent:*ラベルへ一括移行しました。
- issue処理時に使用するClaudeモデルをissueラベル（llm-model:claude-opus-4.6/claude-sonnet-4.6/claude-haiku）で指定できる仕組みをrepositories-managementに実装し、全管理対象リポジトリへ展開しました。
- クリーンアーキテクチャに基づくTypeScript開発で、344テスト/100%カバレッジを維持しながら機能追加を実施しました。
- 複数のAIエージェントが並行稼働する自律型開発環境で人手の介在なく開発サイクルを継続させるため、エージェント完了通知受信時のPRへのnext action date自動設定、LLMモデル・ログパス・設定ファイルパスの一元管理、ネットワーク障害時のリトライ強化を実施し、運用基盤の安定性を向上させました。

2026-05-21 to 2026-05-21

個人開発 セキュリティポリシー強化 at OSSプロジェクト aws-event-mocks (フォーク)

npm, セキュリティ, .npmrc, 供給チェーン攻撃対策, fork開発

- npm供給チェーン攻撃（typosquatting・パッケージ乗っ取り等）対策として.npmrcを追加し、min-release-age=7（新規公開パッケージを7日間待機してからインストール可能）・ignore-scripts=true（インストール時スクリプト実行禁止）・save-exact=true・package-lock=trueを設定しフォークリポジトリのセキュリティポリシーを強化しました。

2026-05-07 to 2026-05-15

個人開発 Androidアプリカスタマイズ開発 at OSSプロジェクト termux-app (HS Termux)

Android, Java, GitHub Actions, CI/CD, APK, Termux, fork開発

- OSSのAndroid端末エミュレータ「Termux」をフォークしてカスタムアプリ「HS Termux」を開発しました。公式アプリとの共存インストールを実現するAndroidアプリID変更、独自アプリ名への改名、インストール障害の修正対応を行いました。
- GitHub Actionsを活用したCI/CDパイプライン構築を行いました。全ブランチpush時にAPKを自動ビルドしてGitHub Artifactsへ保存し、PRの説明文へダウンロードリンクを自動掲載する開発効率化フローを実装しました。
- CI PR説明文に出力するAPKダウンロードリンクをヘッダー+個別リンクの複数行形式から単一リストアイテム形式に簡略化し、PR説明文の可読性を向上させました。
- Android UIのJava実装として、ターミナルセッションパネルの+ボタン押下時にURL・ショートネーム入力モーダルを表示するダイアログをTextInputDialogUtils.java（52行）として新規実装しました。
- Android 15（API 35）でGoogleが施行したdataSync前景サービスの6時間/日実行制限に対応するため、TermuxServiceとRunCommandServiceのforegroundServiceTypeをdataSync→specialUseに変更し、AndroidManifest.xmlへ特殊用途サービス宣言（PROPERTY_SPECIAL_USE_FGS_SUBTYPE）を追加することで、ターミナルセッションの長時間継続稼働を保証しました。

2025-05-12 to 2026-03-31

フルリモート バックエンドエンジニア at 大手メーカーソフトウェア開発プロジェクト向けプロダクトグローバルチーム

Golang, Python, Terraform, TypeScript, Jira, Artifactory, Docker, Cucumber, Gherkin, Gin, FastAPI, AWS, GCP, Azure, GitHub Actions, GitHub Projects, GitHub Issues, Slack, Kubernetes,

- 日本人は自分だけのインターナショナルチームでの開発でした。
- スクラム体制での開発に沿いつつ、リントの強化やCIの調整、リファクタリング、コードベースの改善を推進しました。
- GoとPythonで書かれた既存プロダクトの機能開発を行いました。
- Terraform Providerのメンテナンスを行いました。
- ArtifactoryのAPIを使用して自動でのコントロール機能の拡張を行いました。
- 外部チームとの協業が必要なリリースにおいて、スケジュールや日程の調整を担当しました。全チームが同じ認識を持てるよう、手順や予定の確認、残タスクの洗い出しを行い、スムーズなリリースに貢献しました。

2025-04-22 to 現在

フルリモート インフラ,バックエンド,フロントエンドエンジニア at ロジスティクスSaaSスタートアップ シリーズC

TypeScript, hono, React, Vite, AWS, amplify, GritQL, biome, Devin, copilot, Claude Code, Claude Code Review, Trunk-based development, GitHub Actions, Backlog, GitHub Projects, GitHub Issues, Slack,

- ナレッジ共有・人材育成・評価に使用するSaaSプロダクトの開発でした。週2〜3日程度の稼働でした。
- エンジニア常時1.5人程度、PdM1人のスモールチームでした。
- プロダクト公開から順調に顧客数を伸ばし、1年足らずでARR7,500万円程度に成長しました。
- AIがソフトウェア開発でパワーを発揮できるよう環境を整備し、レビュー負荷を下げる自動的な仕組みを増強しました。最終的にエンジニア1人分のボリュームでありながら、PRの生産スピードは以前の平均1.5件/日から最大50件/日に到達し、開発スピードが約33倍に伸びました。
- カスタマーサポートメンバーの運営上の課題を聞き次第、随時改善のための細かな機能を導入しました。新機能開発や改善スピードの速さについて、営業の方から「改善が追いきれないほど早い」と評価いただきました。
- Trunk-Based Developmentを導入し、参画開始時には2週間に1回程度だったリリース頻度を、長くとも2日に1回、理想的には1日1回を目指せる体制にしました。
- Feature Flagを導入し、大量のPRがマージされる環境でもコンフリクトを極限まで抑え、余計な調整コストを削減する体制を構築しました。
- e2eテストの導入、TypeScript CDKのAWS Stackの構成をstateの観点で3種類に整理、AIを使用した開発を組み込めるよう整えました。
- biomeを1.9から2.0.0-betaに移行し、GritQLでプロジェクト固有のルールを記述して静的解析を充実させました。
- AWS Amplify Gen2にて各作業ブランチの環境を個別に作成し、20程度の環境を維持できるよう整えて動作確認とe2eを確実にする環境を構築しました。
- Webプッシュ通知インフラをAWS CDK Custom Resourceで構築しました。ECDSA P-256キーペアを自動生成してプライベートキーをSecrets Managerに保管し、パブリックキーをLambda環境変数とフロントエンドビルドに注入することで、master・ステージング・レビュー全環境でVAPIDベースのWebプッシュ通知を機能させました。
- マルチテナントPWAのWebアプリマニフェスト動的ルーティングを実装しました。index.htmlにインラインIIFEスクリプトを追加し、ページロード時に`?tenant=`パラメータを読み取ってテナントごとのマニフェストhrefを設定し、複数テナントが独立してPWAをインストールできるようにしました。
- パスキー(WebAuthn/FIDO2)のUXを改善しました。`useGetFeatureFlags`のplaceholderDataから`?? {}`を削除してフィーチャーフラグ取得前にパスキーモーダルが描画されるレースコンディションを解消し、ブラウザ・プラットフォーム情報から「Chrome on Mac」「iOS Safari」等のデバイス名を自動設定して複数のパスキーを区別できるようにしました。
- フラキーなE2Eログインテストの根本原因を3件特定・修正しました。Cognitoの一時的なスロットリングに対するリトライロジック追加、DB/Cognitoユーザー状態デシンクの再同期処理、スタックCI待ち行列によるカスケード失敗の解消、加えてDate.now()タイムスタンプベースのID生成を11テストスイートにわたって固定IDに置換しID衝突起因のテスト失敗を根絶しました。
- CI/CDパイプラインの信頼性向上とコスト削減を行いました。スタックCI実行のキャンセルをforce-cancel APIに切り替え、待ち行列管理ロジックにpending状態の実行も含めるよう拡張し、未使用プレースホルダーワークフローを削除し月約$8のコストを削減、DynamoDBテーブルの不要なPoint-in-time Recoveryを無効化しAWSインフラコストを削減しました。
- API Gatewayの設定からAsyncAPI 2.6.0 WebSocket仕様を自動生成し、手作業なしでAPI仕様書のカバレッジを向上させました。
- セキュリティ観点の私の調査結果に対して内部監査が入り、その結果は、網羅的かつ詳細で指摘できることがない、非常によい品質ですとフィードバックをいただきました。

- PdMの方にAIの使い方を共有し、AIを使ったワークフローに作り変えることでプロダクトチーム全体の生産性向上につながりました。
- AWSのセキュリティ監視体制を強化しました。全リージョンにわたり複数の国際的なコンプライアンス基準（FSBP・CIS v1.4・PCI DSS v4・NIST 800-53）に基づく継続的な自動監視体制を構築し、インフラのセキュリティ状態をリアルタイムで可視化できる基盤を整備しました。
- 動画コンテンツの配信安定性を向上させました。一部のフォーマットの動画が処理できずユーザーが学習動画を視聴できない問題を解消し、動画生成処理が一時的にサービス全体に影響する問題を制御することで、安定した動画配信を実現しました。
- 脅威検知・監査ログ・アクセス権限分析を目的として、GuardDuty（全保護プラン）・CloudTrail（KMS暗号化・7年WORM保持）・IAM Access Analyzer・VPC Flow Logs・S3 Storage Lensを導入しAWSクラウドのセキュリティ監視体制を多層防御で構築しました。
- PlaywrightのE2EテストでwaitForResponse競合状態・Playwrightプロセス予期終了・CIタイムアウトなどのフレーキーネスを体系的に修正し、ユーザータグ・アセスメント・コース管理・NPSエクスポートなどの新規スペック追加も実施してCIの安定稼働を維持しました。
- プッシュ通知のクリック時ナビゲーション修正・通知チェックボックスのUI改善・E2Eテスト整備を実施しプッシュ通知機能の信頼性を向上しました。
- AWS CDK・Route53・CloudFrontを活用して本番PWAドメインのDNS解決とブランチ別レビュー環境デプロイ基盤を整備しました。
- CloudTrail・CloudWatch Logs・Lambda・Secrets Managerを組み合わせてS3セキュリティ変更を自動検知しSlackへ通知するセキュリティアラーム基盤を構築しました。
- ReactのuseWatchスコープ最適化・TanStack Routerへの遷移切替・iOS動画再生フォールバック対応により、モバイル対応とテスト安定化を同時に実現しました。
- 依存ライブラリに潜むパストラバーサル・プロトタイプ汚染・クラウドメタデータ漏洩などの高深刻度脆弱性を集中的に修正し、REST API の認証バイパスとマルチテナント間のデータ分離不備も解消して、アタックサーフェスを大幅に縮小しました。
- PostgreSQL 全文検索インデックスを基盤にコンテンツ横断検索機能をドメインモデル・REST API・React UI の全層で設計・実装してフィーチャーフラグ付きでリリースしました。
- 再帰 N+1 クエリを単一 CTE に置換し、無制限データ取得 API にページネーションを追加し、動画変換処理の高エラー率の根本原因を特定・修正することで、バックエンドの性能と安定性を向上させました。
- スキルレベル形式コース機能を Clean Architecture に沿って Skill / SkillLevelContent / SkillFulfillmentRequirement / SkillCustomStep / SkillApprover / SkillExpiryRule のユースケースとアダプタ I/F から実装し、REST エンドポイント・TanStack Query Hooks・Presentational UI コンポーネント・Storybook ストーリー・E2E スケルトンとレイヤーごとに段階的に構築して並行開発を可能にしました。
- CourseMessage の REST ルート・TanStack Query Hooks・コース受講画面と管理画面活動モーダルの UI・ENABLE_COURSE_MESSAGE フィーチャーフラグ付き E2E シナリオを順次実装し、コースへのコメント機能をレガシーの ItemComment ベースからコース専用実装へ移行して本番接続しました。
- グローバル検索を研修コース・理解度テストまで含むよう拡張しました。searchable_contents PostgreSQL ビューを DISTINCT ON で再定義して下書き保存中の最新バージョンが公開済みバージョンを検索結果から隠す不具合を解消し、フロントエンドのクライアントマージ回避策を撤去してビューを唯一の source of truth に戻しました。
- iOS と Android 双方の PWA ホーム画面追加ダイアログを実装し、Android 専用フィーチャーフラグを汎用 PWA フラグから分離しました。旧 PWA インストールページの撤去（E_LEARNING-862）・URL はみ出し修正・iOS WKWebView の translate 変換に起因する横向き全画面再生のタップ領域ズレ（E_LEARNING-859）を inset-0 + flex 構成への変更で解消する修正も併せて実施しました。

- 翻訳ステータス表示機能（ENABLE_753_SHOW_TRANSLATION_STATUS）を本番デフォルト ON に変更し、E2E の WIP トグル処理を削除して開発完了済み機能を本番と同条件でテストできる体制に整えました。
- 動画レンダリングパイプライン（Remotion 移行）の前後互換性を回復しました。useItemVideoRenderingStatus を新パイプラインの REMOTION_HLS_MANIFEST パスポーリングへ修正し、旧データへの日本語 MP4 フォールバックと VTT アップロード後 60 秒の Grace Period を追加して「変換中...」表示が永続化する問題群を解消しました。
- 字幕翻訳パイプラインの回帰を E2E で自動検知できるようにしました。processing bucket から languages.json が欠落した状態で translation-status isComplete=true を待つ専用シナリオを追加し、過去に staging で発生した throw-on-missing パターンの再混入をリグレッションテストで自動検知する仕組みを構築しました。
- ffmpeg-normalizer WebSocket API の $connect ルートに Cognito ID Token を検証する Lambda Authorizer（REQUEST type）を追加し、フロントエンドから ID Token を `?token=<token>` クエリパラメータで付与するよう改修して未認証 WebSocket 接続を遮断しました。
- Cognito ユーザープロビジョニングの冪等性を担保し E2E のフレーキーログイン失敗を根絶しました。AdminGetUser 存在確認時にパスワードを再同期する setPasswordWithVerification ヘルパーを追加し、Disabled ユーザーの再有効化と UI 経由作成後の API リシンクの構造的欠陥も修正しました。
- 特権アクセスユーザー一覧の 100 件ページング上限を撤廃しました（E_LEARNING-868）。Prisma の take パラメータをオプショナル化し getAllUsers ユースケースを追加して、isIncludeAllOffice=true の管理者画面で全ユーザーを参照可能にしました。
- Amplify レビュー環境の IAM ロールクォータ管理を強化しました。cleanup cron を毎時から 15 分ごとに短縮し、GitHub Actions scheduled run drift（実測 30〜93 分遅延）への対策として PR close 時の pull_request_target 直接トリガーを追加して cancel-in-progress による上書きも防止し、1000/1000 クォータ枯渇によるデプロイ失敗を抑制しました。
- マイグレーションランナーがフィーチャーフラグ同期時に DB の手動設定値を上書きする不具合を修正し、フラグ定義カタログの差分のみを反映するロジックへ改修しました。
- AI エージェント並行多人数開発の運用安定性を底上げしました。core.hooksPath 経由で起動された pre-commit-for-ai.sh が本流リポジトリの diff を worktree のインデックスへ混入させる不具合を GIT_DIR / GIT_WORK_TREE の絶対パス pin と git rev-parse --show-toplevel ベースの解決で根本解消し、共有 PostgreSQL を枯渇させていたフルテスト実行を変更ファイルスコープへ絞り込み、auto-stage 動作の回帰テストも追加しました。
- セルフホスト CodeBuild ランナーの Docker イメージに Google Chrome と Playwright バンドル FFMPEG を焼き込み、ランタイム DL ハング起因の 35 分タイムアウトによる E2E 一斉スキップを解消する基盤を整備しました。
- グローバル検索系を中心に長らく無効化されていた E2E テスト群を決定的待機ヘルパーの刷新によって順次再有効化しました。API-only readiness ゲート（3 回連続成功待機）と空キャッシュ再クエリを導入し、Assessment result / passkey-password-change / subtitle-cache / nps-tenant 等の個別根本原因（stale React Query キャッシュ・real timer 競合等）を特定・修正して復元しました。
- マニュアル一覧 UI の操作残留状態と表示不整合を解消しました。コンテンツ移動・削除後のチェックボックス選択残留、削除モーダルのカラー不整合（E_LEARNING-824）、部分失敗時の一覧未更新、コース詳細サイドパネルのオーナー情報「-」表示を修正し、ページサイズの sessionStorage 永続化（E_LEARNING-856）も追加しました。
- GCP 日次コストレポートが参照する billing export テーブル名を最新フォーマットへ追従し、社内コスト監視の継続性を担保しました。

2025-04-22 TO 現在

フルリモート 技術顧問 at AI営業SaaSスタートアップ シリーズA

TypeScript, Golang, Python, BrowserUse, GitHub Actions, Devin, Cline, Copilot, GitHub Projects, Trunk based development,GitHub Issues, Slack,

- 開発チームにもクライアントの対応を行う体制を整え、顧客価値を中心とした課題にきづける体制を構築しました。

2025-02-27 TO 2025-03-31

フルリモート週3稼働AIエンジニア at 老舗AI企業子会社

Python3, AutoGen, OpenAI, GPT-4o, Gemini, ReactJs, Gatsby, AutoGenStudio, pytest, GitHub Actions, GitHub Projects, GitHub Issues, Slack, Azure, SQLite,

- クライアント企業のマネージャー向けAIAgentプロトタイプ開発プロジェクトでした。
- 1ヶ月という短期プロジェクトでしたが、機能開発の速さ、チームのタスク管理環境整備、CI/CD環境整備、若手メンバーのタスクアサインコントロール、確定的に可能てなくても複数手段を検討して実現させた事などを評価していただきました。
- 他のエンジニアと比べて単価が高いものの、パフォーマンスが充分にあり良いともフィードバックをいただきました。
- 小さく、かつ期間の短いプロジェクトでしたのでコンバクトで最低限の管理と自動化のみ実施しました。プロジェクト継続が決まった後必要な作業に関してはタスクとして残しておき、意図的に後回しにすることを明確にしました。
- LLMにて不確定に振る舞う箇所とLLMから使用されるTool群でテストのポリシーを分け、Tool群には単体テストと統合テストを自動で実施できるようにしました。
- GitHubActionでCI環境を構築し単体テスト、APIの起動、フロントのビルドなど各種の基本的な問題は自動で検知できるようにしました。
- クライアントからのフィードバックが曖昧になり始め、AIAgentで何を実現すべきか迷っているように思えたので具体的なユースケースやシナリオを設定してクライアントのワークフローへの貢献ができるよう努めました。
- 最終報告のデモに向けてAIAgent使用初期のAIの振る舞いと、様々なユーザーの期待を理解した上での振る舞い両方ができるよう実装しました。
- 最終報告時のデモのシナリオや、今後どのように発展させることが可能かなどをクライアントに起きうる変化をベースに考え、資料案を提出しました。

2025-01-01 to 2025-02-28

リモート稼働フルスタックエンジニア兼エンジニアリングマネージャー at SaaSスタートアップの日本市場向け福利厚生プロダクトリプレース

Golang, OpenAPI, AWS Lambda, AWS RDS Aurora, MySQL, SST, React, TypeScript, playwrite, Jest, shadcn, Clean Architecture, DDD, GitHub Actions, GitHub, GitHub Issues, GitHub Projects, Docker, docker-compose, Slack, Swagger, kubernetes, Flutter, ReactNative, Ubuntu

- ピボットにて見つけたチャンスに振り切ることが決定したため、変更に弱く身重になってしまっていたシステムの構造が課題でした。
- バックエンド、フロントエンド、インフラすべてで以前のプロダクトに最適化され必要機能の8倍程度のサイズに膨らんでいた為作り直すことにしました。
- kubernetesを使用していたためメンテナンス労力的にも金銭的にも負荷が高くなっていたのでAWS Lambdaを使うようにしました。
- 既存のシステムではFlutterを使用しましたが、現在のチームメンバーのスキルセットと採用市場の調査結果からReactNativeを使用することにしました。
- 別案件で既存メンバーは忙しい為新たなメンバーを採用し自動化されたオンボーディングプロセスですぐ即戦力として新メンバーにも貢献してもらいました。
- このチームでは私ではなくメンバーのリファラルで優秀なメンバーを採用、3年弱のこのチームで累計30人以上にお手伝いいただいた中で自らやめたメンバーは2人のみにとどまっていて、該当の2人のうち1人は起業もう1人はご家族の看病とのことで、メンバーは現在の開発チーム環境を気に入ってくれていると考えています。
- 一部のロールが既存の方法でなかなか採用できず、既存の方法では滞りが見え始めていたためまずアシスタントを採用しエンジニア採用体制を増強しました。この取り組みは成果があり、30人以上の候補者から4人の優秀な候補者との出会いがあり、メンバーの増強に成功しました。
- プロダクトの性質上技術的にはあまり課題はないもののスケジュールがタイトなため引き続き実装以外の時間を極限まで減らし対応しています。

2024-05-28 to 現在

リモート稼働フルスタックエンジニア兼エンジニアリングマネージャー at SaaSスタートアップ営業支援系AIプロダクト実装

Golang, Python3, OpenAI API, Anthropic API, Gemini, Claude Code, Selenium, AutoGen, AWS Lambda, DynamoDB, AWS SQS, Postgres, AWS RDS Aurora, SST, React, TypeScript, Next.js, Prisma, shadcn, Clean Architecture, DDD, GitHub Actions, GitHub, GitHub Issues, GitHub Projects, Docker, docker-compose, Slack, Jest, Swagger, OpenAPI, Playwright, RESTful, Ubuntu, AWS Amplify, AWS ECS

- R&Dでの成果からプロダクト化を進め、ビジネス要件を踏まえた本格的な実装に移りました。
- プログラミングを安定させるためprogramatic に制御すべき部分に型安全ではないPythonを使わなくてすむようデータやプロセス管理を担うシステムと主にAIにて作業するべき不確定要素の多いシステムを分ける構成としました。
- プロダクトの性質上同時に数千件以上の処理をこなす必要があったため、Serverless構成とし、AWS Lambdaを実行環境として構築しました
- Lambdaの上限は問題なかったもののSQSから実行できるLambdaの数の上限に達してしまいスケールできるサイズがクリップしたためQueueからの実行を独自に実装して回避しました
- 毎月にAIの性能が上がったりコストが下がったりと、使うLLMの選択肢が増える環境にあったため頻繁に各モデルの評価と、成績鍵良いものへのスイッチを行いました。
- 同時期に行っていた採用活動の成果が良くチームメンバーを16人程度に増えたがマネジメントコストが増大したため、できるだけ自動的に、非同期ですべてが把握できるよう体制を構築しました。
- 今では完全リモート、稼働時間自由、定例会議無しで目座ずべきゴールの共有、優先順位の判断ルール、各エンジニアの稼働とスキルの評価の体制を構築しマネジメントにかかる時間をデイリーで2.5時間程度にまで下げられています。1時間以内を目指してまだ改善途中です。
- この時期に導入したデイリーの評価のシステムで今まで検知できていなかった問題が見えるようになりフルフレキシブルな環境でありながら翌日には問題の検知と解消ができるようになりました。
- 個々のエンジニアのパフォーマンスの評価が正確にできるようになったことで単価を上げるべき人、下げるべき人、やめてもらうべき人が定量的な評価時楽で可能となりチーム全体で使用していたコストの効率が体感ではありますが2倍程度になりました。
- プロダクト強化のためエンジニアリングチーム自身でプロダクトを使用するようオペレーションの設計と継続的にプロダクト強化圧がかかる体制化を構築しました。
- 会社の売上発生イベント回数をエンジニアリングチームの目標として設定し、自発的なプロダクト強化体制により営業チームからも良いフィードバックをいただきました。
- 最大20のAIエージェントを24時間365日同一コードベースで並行稼働させ、スキルベースの指示、CI/CD強制、自己文書化するコード構造を通じて1人のテックリードがオーケストレーションする体制を構築しました。手動開発期間(2025年8-11月平均約3PR/日)からAIエージェント導入後(2026年1-3月平均約13PR/日)へと約4倍のスループット向上を達成しました。ピーク時には1日50PRがマージされました。8ヶ月間(2025年8月-2026年3月)で合計1,284PRをマージしました。全PRは自動CI(lint、型チェック、e2eテスト)とコードレビューを経てマージされました。ドキュメントではなくlintルール、CIチェック、自己文書化するディレクトリ・ファイル命名規則による厳格なアーキテクチャ強制で品質を維持しました。
- Lambda関数のARM64(Graviton2)アーキテクチャへの移行とメモリ最適化によりAWSインフラコスト削減を実施しました。全Node.js Lambda関数をARM64に移行し、約20%のコスト削減を達成しました。バイナリ互換性を考慮した関数ごとのアーキテクチャ戦略を設計し、PrismaのARM64バイナリターゲットを追加、アーキテクチャ設定検証のためのCDKアサーションテストを実装しました。初期ARM64デプロイ時のCloudFormationレースコンディション問題を調査・特定し、フォローアップPRで解消しました。

2024-05-28 TO 現在

開発者・メンテナー at OSS開発 (個人プロジェクト)

TypeScript, Node.js, GitHub Actions, GitHub GraphQL API, Clean Architecture, DDD, Jest

- [object Object]
- [object Object]

2024-04-01 to 2024-05-27

リモート稼働フルスタックエンジニア兼エンジニアリングマネージャー at SaaSスタートアップのAIプロダクトR&D

Python3, OpenAI API, Anthropic API, Llama, Gemini, ChromeExtension, Selenium, PyAutoGUI, AutoGen,

- 誰も何をどうできるかわからない中での開発になったため、ボードメンバーとできるだけ頻度高く情報共有を行い、チームでランダム探索を行いました。
- LLM（Large Language Models）の精度が向上したことを背景に、社内向けツールのアイデアを実装しました。新人社員が行うようなタスクの遂行を目指し、自律的に動作するよう構築しました。現在のLLMの精度では、タスクが3-4ステップ以上になるとLLMが途中で必要な操作を忘れたり、依頼していないことを実行してしまう問題が発生していましたがこの問題を解決するために、LLMのスペックやタスクの構造を見直し、最適な構造に作り変えました。現在は、テスト段階で使用したタスクに関しては、体感として90%以上の確率でこなせるようになっています。
- また、タスクの性質により起きがちな間違いが見えてくるようになったため、作業を細かく分けたり、サンプルを示したりすることでLLMのタスクの実行が安定するようになりました。
- OpenAIのAPIだけでなく、他の大手2社のAIも並行して使えるようにシステムを修正しました。一部のタスクに関しては、より安いLLMやより早いLLMを使得るような構造で構築しました。複雑なタスクを複数のエージェントで遂行し、自律的に修正しながら進めるためマルチエージェントのフレームワークを調査し、採用しました。マルチAIエージェントで構築できる状態にしておくと、ある程度複雑な箇所やAI同士でのレビューなど自律的な行動が取れるようになり、汎用的かつ成功確率の高いものにできるようになりました。
- このプロジェクトは4人のチームで取り組みましたが、最初は作業の切り分け方に非常に悩みました。当初はどのような構造になるか全く想像がつかないどころか全員でAI周りは明るくなかったので調査から始めなければならないにもかかわらず、急いで成果を出したいという状況でした。リモートチームであり、メンバーが住んでいる国やタイムゾーンもバラバラでしたのでチーム内で使う仕組みを自立的な動作を期待してより機動的に動ける仕組みにしました。
- 実装直後のツールの動作はアルシュの気色悪さが有りましたので見込み客や投資家向けのデモンストレーションのため、AIが何を考えているかの表示や何をAIが操作しているのかが直感的にわかるような表示を追加しました。説明のためのアニメーションや表示には時間がかかるため、設定によりそれらの動作をオフできるように構築しました。
- プロジェクトでは、ユーザーの使用しているパソコンをOS丸ごとコントロールする機能も検討しました。この機能は、画像情報から現在のタスクに必要な操作を指示できるかどうかを調査することを含んでいました。様々な方法を試しましたが、現在のところはLLM一つでは視覚情報からの指示を出せるまでには至らず、一番成績が良かったのはマルチモーダルのアプローチでした。それでも、ピクセル単位でクリックすべき場所の回答が常に間違っていて使い物にならないことが分かりました。各種の既存の調査の結果も合わせて、20%しか精度が出ないことが分かり、異なるアプローチを試すことにしました。
- 現在は5〜6年前に見かけた、商品をレジに並べている商品の種類や産地などを認識できるような方法を調査しています。この調査は詳しい専門家に手伝ってもらいながら進めています。また、ブラウザをコントロールすることがほとんどのタスクでしたが、画像情報からの指示が現在安定的にできないため、Webの画面を操作することに集中してプロダクトの開発を進めています。
- OSSのChrome拡張機能に、3ステップぐらいのタスクであればうまくいくタイプのツールがあったので、拡張機能を使用して機能の一部を作ってみましたが、既に構築しうまくいっているアイデアとのインテグレートのコストが高かったため、途中で開発を停止しました。
- 単純にすべての情報をpromptに入れるとすぐにtoken数上限を超えてしまう為、各種の今回必要な情報を落とさずtoken数を下げる取り組みを複数行いました。

- 自動操作の指示はソースコードの生成で実現させていましたが、当時の成功率は約60%から70%程度でした。成功率が上がらないため、意図的にAIにとって分かりやすいWrapper関数を用意し、Wrapper内部の処理はプログラマティックに行うよう改善しました。その結果、操作の成功確率は体感でほぼ100%に近い数値となりました。
- 日々新しいLLMやフレームワーク、プロダクトが出てくる状況でしたので、ひたすら使ってみる期間を設け、現在の最新の状態がどうなっているかを理解する期間を意図的に作りました。全員で開発していくうちに、チーム全体で1時間あたり、コストが一番多かったタイミングで1万円になってしまいました。そのため、オープンソースのLLMやGrokなどより新しくコストやスピードを下げる可能性のあるLLMの調査を行いました。
- 2024年5月頃の作業中、料金が高く遅いLLMでしかこなせなかったタスクも多く、早くて安いLLMへの単純なリプレイスは失敗しまし。しかしタスクをさらに小さくしたものであれば成功するということがわかりましたので次のフェーズではより小さなタスクに分割できるよう構造の修正をする予定です。

2022-01-01 to 2023-08-31

技術顧問 at 製造業向けSaaSスタートアップ

チーム開発マネジメント、採用活動支援、技術的負債解消支援、組織構造改善

- 前任の責任者が辞め入社したばかりのリーダーに負荷がかかっていてどのように進めていけばよいかの迷いが大きかった箇所に関してサポートを行いました。
- 10分程度話しただけでも体制的、技術的な大小さまざまな課題が五月雨に出てくる状態で負荷の高さがうかがえるところからのスタートでした。
- まずは思考中唐突に課題を思いついてしまい思考が発散して何も進まなくなってしまっていることを解決するため、頭の中にある課題を粒度、ジャンル問わずすべて洗い出し重要度と緊急度の評価を行いました。洗い出し後、今考えるべきことの絞り込みを行い大事な事から集中して進む状態を作りました。
- 定期的にオンライン会議を行い、進めていることの状態評価と次に進めることの計画を決めました。私の作業としてはどの工程も質問のみで全ての洗い出しや評価、決定はリーダー自身にしていただいていましたので3回程度の会議で容量を得ていただき流れに乗りましたのでこの取り組みは不要となりました。
- 目下の課題が順番に消えていくようになると長期的にどうしたらいいかわからない不安が出てきていたようですので会社として何を目指しているかを理解するため、どのような情報があれば次に自分たちがやるべきことが見えるかを洗い出す取り組みを始めました。また、もし社長にも見えていない状態であればどのような情報があると社長が考えやすいかも考えてから年単位の未来を理解する取り組みも行いました。リーダーと社長のコミュニケーションを変えてから2ヶ月程度で見えてくるようになり年単位のイメージがクリアになりリーダーの迷いが減っていることが観察できました。
- 長期的な方向性が見え、次に発生した問題はタスクの増え方のスピードがキャパシティを超えていることでした。採用活動は進めてはいるものの応募が少なく、想定したチーム拡張スピードより1ヶ月程度遅れ始めていたので面を広げる取り組み、社内の採用のためのリソース確保などの計画を立てました。今までは無料の媒体に出すだけだった採用活動をもっと能動的に進めるアイデアを相談、実行ていただき複数の良いメンバーの入社が確定しました。
- 次にリーダー自身が本来進めたいと考えている作業の為の時間が取れず、眼の前の作業に追われていて長期的な課題に特に手がつけられていない問題の対応を行いました。
- 今のリーダーの作業の時間配分を洗い出すとメンバーのマネジメントや開発の他に障害対応や本来自動で対応できるものの自動化の時間がなく手作業でリーダーご自身で引き受けてしまったり入ったメンバーの作業が簡単になるように準備してから作業依頼している事が多い事がわかりました。採用したメンバーも育って来ていたのでコミュニケーションのパスやメンバーにお願いする作業の範囲を増やし、稼働時間の50%を未来の取り組みに当てる時間として確保できるようになりました。
- メンバーが増え、チームの自走が課題になり始めていたのでチームのマネジメントメンバーにも会議に入っていただき、体制改善のディスカッションと実行を行いました。マネージャーによるチーム内のコントロールはうまく安定して回るようになりました。
- リーダーが入社した当初からの技術的な負債の解消をリーダーが一人で抱えていた課題の解消のため、技術志向のメンバーが入るよう会議体を変更してディスカッションと実行を行いました。
- 理想はリーダーのトップダウンな指示なしにメンバーが自発的に動くこととしたので課題の洗い出しと認識、評価などの一番最初にリーダーにやっていただいたようなことから始めました。ディスカッションの中でメンバーとリーダーとの価値観や課題認識等の差が埋まり最初の課題は3週間程度で解消し、メンバーの視野が広がる効果がありました。

2023-02-01 to 2023-08-31

週3リモート稼働フルスタックエンジニア (開発チームエンジニア 2名) at 教育系SaaSスタートアップのアメリカ子会社にて幼児向けプログラミング教育製品の新規開発

AWS lambda, CDK, Node.js, React, AWS app sync, App sync simulator, DynamoDB, Dynamo local, Dynamo Admin, Postgress, Graphql, Ubuntu, TypeScript, Express, DDD, Clean Architecture, GitHub Actions, GitHub, GitHub Issues, JIRA, confulence, GitHub Projects, Trunk-based Development, Mermaid, Docker, docker-compose, Slack, Jest

- ダッシュボード開発時期に立ち上がった、販売を促すための製品でした。
- 短期間でのリリースが必要だったため、更に人数を減らし2人での開発を行いました。
- 機能要件が膨大だったため、DDDを厳密に適用し正規化に時間をかけ、ほぼ全てのコードを厳密な定義より生成することでリリースに間に合わせました。自動生成コードにはテストも含まれ、不具合発生を抑制できました。
- Graphqlの定義を手書きから、モデルからの自動生成に切り替え、生成ツールはオープンソースとして公開しました。
- チーム規模を最小限に抑えることでコミュニケーションコストを削減し、コーディングに80%以上の時間を確保できました。
- エンティティモデル数45個、ライブラリ化して公開したコードやコメント行を除き、開発開始からリリースまでの4ヶ月間に2人で対応したソースコードは11万行となり、うち4.5万行は作成したGeneratorで自動生成しました。
- 製品は日本の親会社でも販売されることとなり事業の拡大に貢献しました。

2022-11-01 to 2023-01-31

週3リモート稼働フルスタックエンジニア (開発チームエンジニア 3名) at 教育系SaaSスタートアップのアメリカ子会社にて製品の中学校教師向けダッシュボード開発

AWS lambda, CDK, Node.js, React, AWS app sync, App sync simulator, DynamoDB, Dynamo local, Dynamo Admin, Postgress, Graphql, Ubuntu, TypeScript, Express, DDD, Clean Architecture, GitHub Actions, GitHub, GitHub Issues, JIRA, confulence, GitHub Projects, Trunk-based Development, Mermaid, Docker, docker-compose, Slack, Jest

- 日本のスタートアップが立ち上げたアメリカ法人の事業で、オフショア開発チーム8名程度の解散後の少人数で機動的な開発を目的としたチームに参画しました。
- 機能要件が膨大だったため、DDDを厳密に適用し正規化に時間をかけ、ほぼ全てのコードを厳密な定義より生成することでリリースに間に合わせました。自動生成コードにはテストも含まれ、不具合発生を抑制できました。
- Graphqlの定義を手書きから、モデルからの自動生成に切り替え、生成ツールはオープンソースとして公開しました。
- チーム規模を最小限に抑えることでコミュニケーションコストを削減し、コーディングに80%以上の時間を確保できました。
- US時間帯の運用に対応するため、メキシコに滞在してタイムゾーンを合わせ、迅速な障害対応体制を構築しました。
- カリキュラム作成チームのアシスタントメンバーが、進捗管理やCS部門からの不具合報告に自律的に対応できるよう、コミュニケーション体制を整備しました。
- 英語ネイティブメンバーとの初めての協働で生じたコミュニケーション課題に対し、図示や事前テキスト共有など、会議進行方法を継続的に改善しました。

2022-06-01 TO 現在

週3リモート稼働フルスタックエンジニア (初期開発チームエンジニア 6名) at コンシューマ向けフードテックスタートアップ

Scrapy, Spider, Playwright, kubenetes, AWS lambda, AWS Batch, Ubuntu, TypeScript, Dart, Express, OpenAPI, Flutter, Mermeid, Puppetier, gRPC, Golang, Python 3, RedShift, firestore, DDD, Clean Architecture, Microservice Architecture, Monolithic Repository, firebase, GitHub Actions CircleCI, GitHub, GitHub Issues, GitHub Pages, GitHub Projects, Trunk-based Development, Mermaid, OpenSeatch, OpenSeatch Dashboard, Docker, docker-compose, Slack, Jest

- 既存のメンバーですでに構築されたシステムを持っているスタートアップでした。
- 外注していたチームの品質が悪く、チームの入れ替えを検討してご依頼いただきました。
- 同時期に入ったメンバー中心でリプレイスを行いつつ、私の方で新機能の開発を行いました。
- 検索機能の追加のためOpenSearch環境を構築しデータの投入からクエリの最適化、チューニングまで行いました。
- 外部サービス連携のためのRPA開発を行い、手動オペレーションの自動化を行いました。
- 連携先が大量に増えていくことが予想されましたので分割統治できるアーキテクチャを採用しスケールしてもスピードを落とさないコードベースを作成しました。
- フロントエンドはflutterで一人しかいなかった為私でコードレビューを行いました。
- 若いメンバーが多く、体制の変化途中で安定しなかったのと完全リモートチームで情報共有や問題の拾い上げに課題があったので会議体の設計やコミュニケーションフローの改善を行い、リモートでコミュニケーション頻度が高くなくても必要なことが明確になるようにしました。
- 自動テストが全くなかったので私で手を入れたところからユニットテストを追加しベースを作成したので他のメンバーもテストを作成してくれるようになりました。
- クレデンシャルが当時いたエンジニアの個人に紐づいているものが多く、チームを離れたあと様々なものが突然動かなくなったので取り急ぎクレデンシャルを更新してまずは動かしつつ、タスクとして創業者のクレデンシャルを使用するよう積んでおき、チームの変化時に同じことが起きないようにしました。
- ci/cdが全くなかったのでCircleCI, GitHubActionsを導入し手作業の大部分を自動化しました。メンバーも内容を理解し使いこなせるようになったのと、自ら他のCIなども探して試したりしてくれるようになりました。
- モノレポの恩恵を受けるため混乱しがちなCIのファイルを分割統治できるorbを作成しました。他のプロジェクトでも同じ課題を抱えていたのでプロジェクト外で作成しオープンソースとして公開しました。
- トランクベース開発に移行しデイリーで数回のデプロイが継続的にできるよう体制とコードベースを構築しました。
- ドキュメントにOpenAPI, Mermaid, などテキストベースで管理できエコシステムの充実したフォーマットを採用し、ドキュメントとソースコードとの乖離を防ぐよう構築しました。ドキュメントはCircleCIでホストし、生成と同時に適切なメンバーのみが見れるよう権限管理をCircleCIの機能で行いました。
- ドキュメントへのコードベースからのリンクが必要ですがCircleCIでホストしている成果物のURLが毎回変わってしまうのでGitHub Pages の機能でリダイレクトを設定し、リダイレクトの先を自動コミットしてドキュメントが自動で最新に保たれる仕組みを構築しました。
- 若いメンバーが実装で止まってしまうことが多く、ケアを行いました。
- 資金調達に難航する時期がありましたのでコスト、稼働調整、体制などについての提案を行いました。
- チーム拡張タイミングには優秀でコミュニケーション能力の高いグローバル人材を採用し、複数のタイムゾーンのリモートチームにも関わらず効率の

良いチーム構築を行いました。

- 高額になり始めていたプロキシ代を節約するため、インフラの再構築を行いました。
- メンバーの行動範囲を徐々に広げるため、ロールの違うメンバーと少しだけやり取りが必要なタスクを徐々に増やしスムーズに自走する体制を構築しました。
- リモートチームなのでリアルタイムなコミュニケーションを前提とせずとも現在自分たちがフォーカスするべきこと、タスクの優先順位、何が起きているかのお互いへの共有ができるだけ少ないコストでできるよう正規化した情報のやり取りをするよう仕組みを作り、浸透するようデイリーのルーチンを組んで実行しました。
- １つの大きなモノリスのサービスとして構築されている部分が明らかに機動力を抑えていたので分割のロードマップを作成しユーザーへの価値提供を強化しつつ、ビジネスサイドの新たな取り組みを実現させながら合わせてリファクタを実行する手順を整えました。こちらは現在実行中です。スタートアップであるため頻繁にビジネスプランや緊急度が変わりますが、週単位での変化では少しの組み換えだけで同時に対応できる状態を維持しています。

2022-03-01 TO 2022-07-31

プロジェクトマネージャー (開発チーム 5名) at ECサイトフルスクラッチ構築案件

FlutterFlow, Firebase, GitHub, Lark, CircleCI, GitHub Actions

2021-02-01 TO 2021-08-31

週1-3リモート稼働Androidアプリエンジニア at 実店舗向け広告SaaSプロダクト

Android, Kotlin, GitHub, GitHub Actions, Testing Automation, GitHub Projects, Confluence, Slack, Mockito

- Androidアプリケーションの設計から実装までの技術支援を担当しました。
- テストが一切なかったため、Mockitoを使用した単体テストの整備とGitHub Actionsを活用したCI/CD環境の構築を行いました。

2022-02-01 TO 現在

技術顧問 at コンシューマー向けアプリスタートアップ

Flutter, Dart, AWS, MySQL, Docker, docker-compose, GitHub, Slack, Terraform Cloud, JIRA, Confluence

2021-12-01 TO 2022-03-31

技術検証兼プロジェクトマネジメント at 研究成果のSaaS化

MATLAB, Python, GitHub, CircleCI

2020-12-01 TO 2021-03-31

週0.5リモート稼働フルスタックエンジニア兼プロジェクトマネージャ (エンジニア2-3名, アシスタント兼QA1名) at ハードウエアから取得した画像をクラウドで分析するMVP開発

Ubuntu, TypeScript, Dart, Express, FlutterFlow, Flutter, Mermaid, no-code, firestore, DDD, Clean Architecture, Microservice Architecture, Monolithic Repository, firebase, fireauth, firebase functions, GitHub Actions, CircleCI, GitHub, GitHub Issues, GitHub Projects, github-flow, Slack, Jest, Trunk-Based Development

- 分析システムをブラウザから使えるようにするプロジェクトでした。
- イテレーションをできるだけ早く、できるだけたくさん回すため、できるだけ労力とコストをかけない体制を構築しました。
- フロントにFlutterのコードを出力できるノーコードを選択し、ビジネスサイドメンバーやアシスタント、デザイナーが直接画面を更新できる体制を目指しました。
- エンジニアリング観点無しで画面の変更ができるため、CIでの自動的なチェックで安全性を担保するようにしています。
- 技術者が私だけだと工数が取れないことと取れたとしても継続性に問題があるためアーキテクチャ設計とバックエンドに強みのあるエンジニアを採用しました。
- 確実に優秀な人を採用するため、アシスタントサービスを導入して大量の候補者の中から選ぶ体制を整えました。
- 海外在住のエンジニアのため、スキル比で換算し40%程度にランニングコストを抑えることができました。
- フルリモートのグローバルチームになるため各種ルールと翻訳環境の整備を行いました。
- ドメインの議論と設計をビジネスサイドメンバーと実施し、会社の拡張方向に合わせた抽象化を行いましたので会話の混乱を少なく抑えることができました。
- FlutterFlowと相性の良いfirebase前提で構成し、FlutterFlowのポテンシャルをフルに活かし最速で完成させる方向を目指しました。
- ビジネスサイドメンバーがMVPを定義から正しく認識できていたため、正しくスモールスタート切ることができています。
- firebase function に依存しないよう、express を抽象レイヤーとして噛ませる設計にし、同時にOpenAPIのジェネレートができる構成を取りました。

2020-03-01 TO 2022-02-28

週2リモート稼働フルスタックエンジニア (初期エンジニア3名から最大35名規模) at 都市開発系SaaSスタートアップ開発支援

Ubuntu, TypeScript, Dart, Express, Flutter, firestore, Postgres, Google CloudSQL, Clean Architecture, Microservice Architecture, Monolithic Repository, Feature Flags, firebase, fireauth, firebase functions, firebase config, Prisma 2, OpenAPI, Docker, Docker Compose, CloudSQL, Google App Engine, CircleCI, GitHub, GitHub Issues, GitHub Projects, github-flow, Zapier, Slack, Jest, Jasmin, Trunk-Based Development

- 既に運用されているモバイルアプリに CI/CD 環境がなかったので構築をお手伝いする案件でした。
- 自動でのデプロイから整備し、インテグレーションテストを整備しました。
- 既に運用されていて品質担保が急務だったこと、テスタブルなコードに変更の必要があったため単体テストをあとまわしました。
- 働く時間も場所もバラバラでタスクの話を slack で行っていたので情報が散らばり、作業の優先順位もつかず無駄なコミュニケーションで前に進めなくなっていましたのでタスク管理ツールで情報を整理する環境を整備し浸透・規律が維持できる体制を整えました。
- 当時 TypeScript 系で一番型安全な ORM が Prisma に見えたので Prisma を導入しました。
- モノレポのメリットを得るためモノレポ構成に移行しました。CI にかかるコストがクリティカルになってきたため差分のある箇所のみ job を実行するよう変更しました。
- 会社が急拡大しパートタイムメンバのみでの実装がビジネス要求に追いつかずトラブルが噴出していた為、海外のエンジニアを採用し海外メンバ��を中心としたチームビルドを行いました。
- 優秀な技術者と出会うため、大量の候補者と面談しテストをする必要があったことと、開発以外の作業も人手不足で大量にスタックしていたためオンラインアシスタントサービスを導入し、採用のための作業にレバレッジをかけ、社内の作業負荷を減らす体制を構築しました。
- 会社の方針として広い方向に拡張していく予定があることと、積極的に新しい技術を取り入れやすくするため、ドメインの設計を行いマイクロサービスアーキテクチャに移行しました。
- マイクロサービスアーキテクチャでの開発を早くするため、新規サービス用のテンプレートを用意しました。運用中のサービスと同じくテンプレートサービスも自動テスト実行や自動デプロイで品質を担保していたため、立ち上がりはかなり早くできていましたが新しい技術を採用する余地を奪ってしまう結果になってしまいました。
- 国籍や稼働している物理的な場所が多岐にわたるため、リアルタイムのミーティングの時間や timezone などのルールの整備を行い、1on1 やスクラムイベントの整備など開発をスムーズに進める環境の整備を行いました。
- 採用に成功したあとはアシスタントチームに開発業務にも関わっていただきマニュアルテストやオンボード、各種ツールや手続き、コミュニケーションのハブになってもらう体制を構築しました。
- 人数が増え全員の目線合わせやデリバリーにコミットする意識が薄れ始めていたので体制を変更しスクラムの経験豊富な PM を採用して少人数のチームを作っていく体制に変更しました。
- 仕様を決めきらずにデッドラインだけが迫る状態が続いていたこと、ビジネス課題の整理をできる人が居なかったこと、PM ロールで入ってくれた方がうまくワークしていたこと、私一人で全体の動きが見えないサイズになっていたことをふまえ、 PM ロールの人数を増やしビジネスレイヤの曖昧さから排除し私が居なくても回る体制を構築しています。
- 各エンジニアの状況を私が把握できなくなってきていたので 1on1 は PM と私の 2 人とも全員と行うよう変更し、目下プロジェクトとエンジニア自身の課題は PM と話し、より長期的な事、興味の方向性やスキルを磨きたいところなどの把握は私で行ってこの会社で働く価値を高く保つよう努めました。
- 日本人の平均的なエンジニアよりも優れた人の採用に 1 ヶ月程度で成功するようになったこと、コストが日本人採用時に比べ 1/3 程度で運用できてい

ること、システムをビジネスの理念に合わせた設計にできていることを評価していただきました。

2020-02-01 to **2020-12-31**

週4リモート稼働フルスタックエンジニア (開発チーム8名) at 大手家電メーカーデータレイク基盤構築

AWS CloudFormation, AWS Redshift, AWS Batch, AWS CodePipeline, AWS CodeBuild, AWS CodeCommit, ShellSpec, Bazel, Docker, BitBucket, JIRA, Slack, Bats, Ubuntu, SQL, Trunk-Based Development

- 既に運用されているデータレイクの改善案件でした。
- デプロイやテストを手動で運用されていたので自動化を行いました。
- AWS 環境かつ Atlassian に寄せたくなかったので AWS CodeBuild, CodePipeline, CodeCommit で自動化を行いました。
- データレイクでのデプロイやテストは普段のアプリケーション開発と違う考慮事項が多くあり面白い案件でした。
- インテグレーションテストでは使うデータが大量なので s3 にバージョンごとのデータを配置しテストコード内でデータのバージョンを指定するよう構築しました。
- 扱うデータは、製品センサーデータ、会計システム、在庫管理システムなど複数の基幹システムからの大規模データを統合的に収集・分析するものでした。
- モブプログラミングをよくするチームで比較的会話が常にあるチームでしたのでトランクベースの開発を提案して実験させていただきました。
- 処理の一部として Shell が大量にあったのでテストフレームワークに ShellSpec を採用し Bazel でビルド・テスト実行するサンプルを作成しました。
- 技術選定では、チームの既存の技術スタックとの親和性を重視し、現在の技術からスムーズに移行できる選択を心がけました。

2019-09-01 to **2020-01-31**

週3稼働フルスタックエンジニア (開発チーム4-6名) at 新規サービス開発・構築

Linux, Ubuntu, Alpine, TypeScript, Node.js, koa, TypeORM, MySQL5.6, Redis, GitHub, CircleCI, Slack, Docker, docker-compose, OpenAPI, AWS ECS Fargate, AWS ElastiCashe, AWS RDS, GitHub projects

- 技術選定から対応させていただきました(言語, FW, middleware, ライブラリ, ツール等)
- バックエンドの設計・実装・テスト・コードレビュー・DevOps を行いました
- I/F 設計は RESTful, アプリケーション内は CleanArchitecture, 開発フローは GitLab flow, DDD, TDD で進めました
- チームビルド・改善を進めました(1on1, KPT, 勉強会開催など)
- オフショアチームの対応を行いました(daily check-in mtg, 設計の共有, タスクのコントロールなど)
- ビジネスレイヤの取りまとめの作業もありましたがこちらはうまく行きませんでした

2018-11-01 TO 2019-07-31

週3稼働インフラエンジニア (アプリケーションエンジニア3名, 社員20 名程度) at AI SaaSスタートアップ企業のアプリケーションインフラ環境構築

Linux, Ubuntu, AMI, HCL, MongoDB, Redis, GitHub, CircleCI, Slack, Docker, Docker Compose, Ansible, Terraform

- 急成長のため、アプリケーションエンジニアが一人で組んでいたシステムのインフラ環境の整備を行っています。
- データベース冗長化のため MongoDB の Atlas への移行を行いました。
- キャッシュサーバーを ElasticCache にて新規に構築しました。
- アプリケーションのサーバーをスケーラブルになるよう設計しました。
- 具体的には AWS の EC2 に構築されていたアプリケーションを Docker に置き換え、 AWS ECS (Fargate) に載せ替えています。
- また、新しい環境はすべて Terraform にて構築を行い手作業を禁止し、いつでも消せる・いつでも作れることができる安全な環境になるよう構築しています。
- 今回は Serverless で開発できる環境ではありませんでしたので使用感のみ Serverless フレームワークと同じになることを目指しています。
- 今回はインフラ専用リポジトリを作成いただき、インフラのためのコード化が自由にできる状況を用意していただいたので、CirlceCI 上で各コンテナの自動チェックやインフラコードの自動テストを行い手間の削減と再現性、変更時の安全性を担保しています。
- コーポレートサイドから要望のあった slack bot を探していたのですがマッチするものがなかなか見つからなかったので 新しく開発し、喜ばれました。

2018-09-01 TO 2018-12-31

週1稼働インフラエンジニア (開発チーム 3 名) at スタートアップ企業の AI 組み込み環境構築

Linux, Ubuntu, Python3, GitHub, CircleCI, Slack, Docker, Ansible, AWS SageMaker, Jupyter Notebook, Terraform

- 機械学習を用いたエンジンを作れる企業に手伝っていただき、エンジンの組み込み部分とエンジン自体の精度を上げるため A/B テストの環境をつくりたいとのことでご依頼いただきました。
- AWS SageMaker を使いたいとの要望があったのと今後の可用性面で適していそうでしたので SageMaker での構築を行っています。
- インフラ環境の全体は terraform で構築し、まだ terraform が対応できていない SageMaker 部分は AWS client を用いて構築しました。
- 開発の環境を Ansible と docker 双方でできるよう、docker image を Ansible で構築するようにしました。
- 今回はアプリケーション構築ではなくインフラでしたが CI/CD を導入し AI エンジンのコードが git に push され次第学習が進むよう構築を進めています。

2018-03-01 TO 2019-07-31

週3稼働サーバーサイドエンジニア (開発チーム 6 名, うちサーバーサイド 1 名) at IoT 企業のバックエンド開発

Linux, Ubuntu, Python2, Python3, Flask, Flasgger, DynamoDB, GitHub, CircleCI, Slack, Docker, Serverless, Ansible, Swagger

- python で AWS Lambda を使用した API サーバーを管理しやすい形に再構築する案件でした。
- FW を使わず Lambda, API Gateway のバックエンドが構築されており、API を使用した自動化をしている状態でしたがメンテナンスがしづらいので Serverless FW を導入したいとのことでした。
- 設計はシンプルではあったのですが外部サービスへの依存が大きくテストしにくい構成になってしまっていて、現に全くテストがない状態でしたので再設計と自動テスト、自動デプロイなどの CI/CD 導入を行いました。
- 新しい設計にクリーンアーキテクチャを採用したことで、ドメインレイヤでのユニットテストではカバレッジ 100 を達成しました。
- ドキュメントがあまりない状態でしたので FW と Swagger を連携し CI にホストさせてドキュメントとしたのと、シーケンス図などのマークダウンで表現しづらい資料は保守性のため html と js でレンダリングできるよう作成し誰でも閲覧できかつ変更しやすい状態になるよう努めました。
- python2 から python3 へ移行し、type hint を導入したのと mypy での解析で不具合を防ぐよう取り組みました。
- Ansible で Ubuntu, Mac のローカルの開発環境構築自動化を行いました。

2018-03-01 TO 2018-10-31

週4稼働バックエンドエンジニア (開発チーム 25 名) at 広告配信サービスの新規開発

Linux, Ubuntu, Golang, Echo, Goswagger, MySQL, Aerospike, Redis, GitHub, CircleCI, Slack, Docker, SchemaSpy, Ansible, DockrCompose

- golang で広告の配信サービスと配信内容を管理する管理画面を構築する案件でした。
- すでにベースが出来上がっている状態からの join でした。
- 後半は管理画面の API サービスを再設計するタイミングができたのでクリーンアーキテクチャを採用しテスタブルかつ仕様変更しやすい設計で構築し直しました。
- swagger や docker での開発環境がなかったため構築して導入しました。
- 若いメンバーも多くチームの設計、開発ツールの使い方、開発フロー関連の知識の底上げが必要と感じていたためランチ帯や業務後でのスキルアップを図るイベントをいくつか行いました。
- 開発環境構築の資料やスクリプトなどが一切なかったため、Ansible で Ubuntu と Mac 用の playbook を作成し、メンバーに共有しました。

2018-02-01 TO 2018-02-28

週4稼働バックエンドエンジニア (開発チーム 2 名) at 新規サービスのユーザー向け Web 画面開発

Linux, Ubuntu, Alpine, Kotlin, spark, exporsed, MySQL, GitHub, Backlog, CircleCI, Slack, Swagger, API Blueprint, Gradle, Docker, AWS, SchemaSpy

- Kotlin で API サーバーを構築する案件でした。
- 途中まで開発したものがある状態でしたが CI や自動デプロイの仕組みがなく、API 設計や DB 設計が不確定な段階でしたので開発の続きも含め、CI の導入や環境の整備からお手伝いしました。
- データベース設計書もなく別チームのコミュニケーションに問題が発生し始めていたのでリバースエンジニアリングで設計書を作成し、CI で継続的に最新のドキュメントがチーム内の誰でも参照できる環境を作りました。
- フロントエンドのメンバーが足りていなかったため js 周りの調査やエラー解消を行いました。
- 内製ではリスクが高いとの判断から開発は外注することになり内製チームは解散になりました。

2017-03-01 to 2017-09-30

週5稼働, うち週3リモートフルスタックエンジニア (開発チーム 6 名, うちサーバーサイド 2 名) at **広告クリエイティブ制作SaaSベンチャー企業のシステム刷新**

Linux, Ubuntu, CentOS, TypeScript, Elixir, nodejs, express, phoenix, Postgres, GitHub, ZenHub, CircleCI, Slack, Swagger, ansible, mix, TravisCI, Codebeat, Codacy, Docker, Docker Compose, AWS EC2

- 要件定義から言語選定、設計、開発、テストまで
- 既存のシステムからの刷新
- 既存システムの一部を機能を見なおして別システムとして構築
- 途中から既存システムの本体も新しく作りなおす事になり、片方のシステムは言語選定から行い、もう 1 つのシステムは開発要因としてお手伝いをしています。

2016-11-01 to 2017-02-28

週3稼働フルスタックエンジニア (開発チーム 4 名, 開発担当 3 名) at **広告クリエイティブ制作SaaSベンチャー企業のシステム開発運用支援**

Linux, Ubuntu, CentOS, JavaScript, CSS, Node.js, LoopBack, AngularJS, MongoDB, GitHub, CircleCI, Slack, Swagger, GitHubProject

- 担当フェーズは開発、テストまで
- 広告の制作支援システムの追加機能開発・運用支援

2016-08-01 to 2017-02-28

週2-5稼働サーバーサイドエンジニア (開発チーム 4 名, 開発担当 1 名) at **IoT SaaSスタートアップサーバーサイドアプリケーション開発**

Linux (Ubuntu), Scala, sbt, play2, MySQL, Ansible, GitHub, CircleCI, Slack, Asana, Swagger

- 4 月から手伝っているプロジェクトでハードウェアの量産経験がある方のジョインと、量産が見えてきたタイミングでサーバーサイドの開発が取り掛かれるようになりましたので、今までのデモ用システムとは別に新しくシステムの構築を開始しました。
- 今後の展開を考え、構成を大きく分離し最終的な目的に多くの手段を取れるよう機能的に分離し、粗結合のわかりやすいシステムになるよう構築しました。
- 意識してはいませんでしたがマイクロサービスアーキテクチャのような構成になりました。
- 担当フェーズは要件定義、言語選定、設計、開発、テストです。

2016-06-01 to 2017-09-30

週2稼働, うち週1リモート Android アプリエンジニア (要員 20 名程度, Android アプリ開発担当3 名) at **IoT デバイス連携 Android アプリ運用**

Linux (Ubuntu), Scala, sbt, Android, CircleCI, GiHub, DeployGate, GitHubIssue, Slack, Beacon, Bluetooth, BLE, CleanArchitecture

- Bluetooth Classic, BLE を使用した通話デバイスと連携するアプリのデザイン更新、機能追加、 不具合修正を行いました。
- Bluetooth, Scala, 音声を扱うプロジェクトで面白い課題が多いプロジェクトでした。
- Scala での Android 開発は初めてですので多くのことを学ばせていただきました。
- 外国人メンバーが多く、自然に英語でのやり取りが発生していましたので、コメントや資料、slack での会話は認識の齟齬が発生しないよういつも以上に注意しました。
- 担当フェーズは調査、開発、テストです。

2016-04-02 to 2016-05-31

週3リモート稼働 Android アプリエンジニア (要員 3 名程度, Android アプリ開発担当 1 名) at IoT ビーコンデバイス連携 Android アプリ基盤変更対応

Linux (Ubuntu), Java, Gradle, Fabric, Android, CircleCI, GiHub, PivotalTracker, Slack, Beacon

- ビーコンデバイスの基盤変更に伴い開発済みの既存の Android アプリの修正
- BLE の Service, Characteristic などを新基板に合わせて変更しました。

2016-4-01 to 2016-07-31

週4-5稼働エンジニア (要員 3 名程度) at IoT SaaSスタートアップデモ用サーバーサイドアプリケーション開発、ハードウェア開発

Linux (Ubuntu), javascript, Arduino, 3DCAD, nodejs, AngularJS, TwitterBootstrap, SQLite, MySQL, npm, bower, Grunt, Sequelize, Ansible, GitHub, Slack, Asana, Swagger

- IoT スタートアップの製品の Web サーバーアプリケーション開発、ハードウェア試作品開発
- 2015 年 12 月から隙間の時間で手伝っていたサービスが本格的に進むことになり、オファーをいただいたので社員になりました。
- ハードウェア側は 3D プリンタや切削、金属加工でのプロトタイプ開発を担当させていただいています。
- ソフトウェア側はデモ用システム設計、データベース&API サーバー、管理画面の開発を行っています。
- サービス、仕様のまとめと量産に向けて技術的な調査、仕様の優先順位の検討、検証用ハードウェアの設計開発などを担当させていただいています。
- プロジェクト管理や資料管理など、開発体制の構築も進めています。
- 担当フェーズは調査、設計、開発、テストです。

2016-03-04 to 2016-03-31

週3リモート稼働サーバーサイドエンジニア (開発チーム 5 名, サーバーサイド 1 名) at センサーガジェットのログ収集・可視化サービス

Linux (Ubuntu), javascript, html, nodejs, npm, bower, Grunt, MySQL, Sequelize, CircleCI, GiHub, Swagger, Slack

- クライアントのガジェットを使用してセンサーデータを集めて保持し、可視化するサービスの開発
- クライアントの新しいサービスとして、営業してみるために使用したいとのことだったので、通常の流れで開発するよりは認証などサービス特有でない部分を飛ばし、機能をデモできるようにするところを急ぎで作る必要がありました。
- データベース設計 & API サーバーの実装と、ガジェットの SDK が公開されているのでサーバーにデータを送るようカスタマイズするところを担当しました。
- 時間がなく、デザイナーやフロントエンドエンジニアもチームに居ましたのでクライアントとサーバーサイドは API のみの粗結合になるよう努めました。
- 集計データを表示するニーズもありましたのでサマリの方法やタイミングについて、運用負荷がかからないよう工夫しました。
- API 仕様は最初の段階で Swagger で提供し、クライアントサイドの開発が早く進むよう努めました。
- 担当フェーズは調査、設計、開発、テストです。

2016-01-16 to 2016-02-28

週3リモート稼働サーバーサイドエンジニア (要員 3 名程度, サーバーサイド 1 名) at データ可視化系スマートガジェットの認証サービス開発

Linux (Ubuntu), javascript, html, nodejs, npm, bower, Grunt, mp2, SendGrid, KiiCloud, GoogleCloudPlatform, CircleCI, sinon, Jasmine, Mocha, CircleCI, GiHub, GitHub Issues, Slack, TwitterBootstrap

- サービスのユーザー登録機能と画面、アプリのログイン機能用 API、ユーザー情報更新や有効性確認部分の構築
- 数社で 1 つのシステムを作る、大規模なクライアントの案件でした。
- システム全体の一部の機能のみを複数社で作り、各社のやり取りは元請けを通して連絡をとっていたため担当範囲外の細かいところまでは把握しづらく、開発に使える期間が短かったので、手戻りがないよう慎重にすすめました。
- 先にこちらから API 仕様を提案したり動く環境を早めに用意して連携先に使っていただくなど、もし認識の違いがあった場合は早い段階で気付けるよう努めました。
- API 仕様ドキュメントは、Swagger を使って実行できるドキュメントを提供し、API を試しやすく動かし方が厳密にわかるよう、開発時に詰まらないよう工夫しました。また、Swagger は開発中スピードや検証検証サイドでの実行も簡単になり、全体の工数削減に協力しました。
- ストレージや環境、構成は指定されており、指定された BaaS が未経験だったため、調査や動きの確認、その BaaS を使用する場合には適する設計を考えるなどで工数がかかり、作る機能に対し普段よりも大きく時間がかかりました。
- 厳しいスケジュールだったのですが納期には間に合い、先方に喜んでいただけました。
- 担当フェーズは調査、設計、開発、テストです。

2015-12-01 to 2016-03-31

週1-3 フルスタックエンジニア (要員 3 名程度) at IoT スタートアップ サーバーサイドアプリケーション開発、ハードウェア開発

Linux (Ubuntu), javascript, javascript, nodejs, AngularJS, TwitterBootstrap, SQLite, npm, bower, Grunt, Sequelize, Ansible, GitHub, Slack, Swagger

- IoT スタートアップの製品の Web サーバーアプリケーション開発、ハードウェア試作品開発
- 初期の頃はサービスのデモ用プロトタイプで使用する API、サイト(or アプリ) の相談や設計、開発を手伝いました。
- 担当フェーズは調査、設計、開発、テストです。

2015-12-02 to 2016-01-29

週 3 日リモート勤務 Android アプリエンジニア (要員 4 名程度, Android アプリ開発担当 1 名) at ソーシャルイベントサービスの Android アプリ開発

Linux (Ubuntu), Java, Gradle, Android annotations, Android Spring REST client, CircleCI, Git, DeployGate, ChatWork

- イベントを企画し、メンバーを募って実行する新サービスの Android アプリ開発
- 企画、デザインは終わっており、データベース、API サーバー、iOS アプリは Android アプリと並行して開発だったため、できた API から随時アプリに組み込む流れで開発しました。
- Android アプリ自体はほとんどがデータの表示のみのため、シンプルな構成のアプリになりました。
- リモートでの開発だったことと、先方に Android の技術者がいないとの事だったので毎日作業進捗はソースコードの push 以外にも Chatwork への文章での報告や作業予定、見込みを報告しました。
- CI や作業途中のアプリを見る仕組みは考えていなかったとのことでしたので CircleCI を導入し DeployGate へアプリを自動で更新させるようにし、常に先方が状態を把握できる環境を作るよう努めました。
- 担当フェーズは設計、開発、テストです。

2015-11-06 to 2016-01-15

週 3 日リモート勤務 フルスタックエンドエンジニア (要員 3 名程度, 開発担当 1 名) at サービス連携サイトプロトタイプ開発

Linux (Ubuntu), javascript, html, nodejs, AngularJS, mysql, npm, bower, Grunt, CircleCI, GiHub, GitHub Issues, Slack, Sequelize

- ユーザ指定の条件で既存サービスを連携するプロトタイプ開発
- クライアントの自社サービス予定で、本実装の前の、機能のみ試すプロトタイプ開発でした。
- どんな機能があると良いかや、どんな画面が必要かなどを考えるところからかかわらせていただきました。
- まずは機能を試したいとの事だったので、本番では採用しないが早く作れる方法を採用して構築しました。
- 担当フェーズは調査、設計、開発、テストです。

2015-10-26 to 2015-11-05

週 3 日リモート勤務 Android アプリエンジニア (要員 3 名程度, Android アプリ開発担当 1 名) at データ収集 IoT ガジェット連携 Android アプリ開発

Linux (Ubuntu), Java, Gradle, Android annotations, Android Spring REST client, CircleCI, GitHub, Deploy Gate, GitHub Issues, Slack

- 遠隔でガジェット周辺のデータを取得し、表示する Android アプリの開発
- 途中までアルバイトのエンジニアの方が作ってくれていたのですが、出勤可能な時間的に先方が予定していたデモに間に合わない可能性がありましたので私が引き継ぐことになりました。
- iOS アプリの開発はすでに進んでいて、仕様は「iOS アプリと同じ」と明確で分かりやすかったので仕様確認のための時間消費や認識違いなどのトラブルは起きませんでした。
- 10 日程度しかなくライブラリやフレームワークなどの開発を早くする環境が整っていませんでしたので、新しくプロジェクトを作成しすでにある機能を移植する形で引き継ぎました。
- 時間がない中での開発でしたが期限に間に合い、クライアント様に喜んで頂きました。
- iOS アプリの方ではガジェットとの連携に問題が生じていたので Android 開発中に得られた情報を使って調査に協力しました。
- 担当フェーズは調査、設計、開発、テストです。

2015-09-14 to 2015-10-25

週 3 日リモート勤務 Android アプリエンジニア (要員 4 名程度, Android アプリ開発担当 1 名) at IoT 製品連携 Android アプリ BLE 技術検証開発

Linux (Ubuntu), Java, Gradle, Android annotations, Android Spring REST client, Beacon, CircleCI, Git, PivotalTracker, Slack

- Beacon デバイス Farmware 確定のための Android アプリ側技術検証
- 対応 OS が Android4.0 以上だったため 4 と 5 の両方検証し、プロトタイプのアプリを作成しました。
- Beacon のアプリを作るのは初めてだったため、調査からさせて頂きました。信号の内容や読み取り、書き込み、通知についてとても勉強になりました。
- ガジェット側の場所を特定するためのアプリだったため常に信号受信する必要があったので、バックグラウンドでの動作確認や OS にプロセスを止められた時の再起などの検証も行いました。
- 時間がない中での開発でしたが期限に間に合い、クライアント様に喜んで頂きました。
- iOS アプリの方ではガジェットととの連携に問題が生じていたので Android 開発中に得られた情報を使って調査に協力しました。
- 担当フェーズは調査、設計、開発、テストです。

2015-05-01 to 2015-08-31

週 3 日 フルスタックエンジニア (要員 1 名) at ハードウェアスタートアップ製品開発支援

Linux (Ubuntu12), arduino, C++,

- マイコン、タッチパネル、bluetooth を用いた製品のプロトタイプ開発
- ハードウェア開発の技術が未熟な中で関わらせていただき、とてもありがたい環境でした。
- 技術的に未経験のことが多く、基本的なところから調べながら進めるプロジェクトでした。
- 直接は使いませんでしたが CAD, 3D プリンタ、切削等工作機械の勉強もさせていただき、製品開発時の技術の選択肢が広がりました。
- 担当フェーズはマイコンのプログラム設計、開発、回路設計、開発です。

2015-01 to 2015-10

週 2-4 日 フルスタックエンジニア (要員 13 名程度, 開発担当 7 名, うちサーバーサイド 1 名) at ハードウェアスタートアップ企業開発支援

Linux (Ubuntu12, Ubuntu14), arduino, C++, javascript, nodejs, AngularJS, TwitterBootstrap3, MongoDB, Jenkins, Git, Ansible, Yeoman, Apache2, Julius, OpenCV, Slack, Redmine, Swagger, elasticsearch, fluentd, kibana

- IoT の新製品開発
- ハードウェア、電子回路未経験の中で関わらせていただき、とても勉強になりました。
- 開発プロジェクトの経験がある中で時間的に一番長く出勤していることもあり、基本の設計とエンジニアのタスク管理やスケジュール調整などの管理面も関わらせて頂きました。
- 各種センサー、音声認識、画像認識等の未経験の技術を使用していること、エンジニアほぼ全員がフルタイムではない働きだったこと、学生さんも多くチーム開発のノウハウやツールに慣れていないチームでしたので、スケジュールの調整やタスクの割り振り、情報共有、開発体制・フロー構築、運用自体に苦労しました。
- 変更が多く、フルコミットでないメンバーで回す必要があるため、わかりやすく分離しやすい設計と柔軟な開発体制を保つこと、メンバーのスキル向上を意識しました。
- 担当フェーズはサーバーサイド設計、開発、ハードウェア回路設計、開発、筐体内ソフトウェア設計、開発です。

2014-09-01 TO 2015-04-30

開発フルスタックエンジニア (要員 4 名程度, 開発担当 1 名) at 業務システム開発

Windows, Linux, Scala, Java, Javascript, Gradle, Skinny (Scala), Flyway, Jetty, Tomcat, Android annotations, Android Spring REST client, TwitterBootstrap3, AngularJS, jQuery, Robolectric, Postgres9.4, Jenkins, Git, Redmine

- Android とブラウザで使用する業務システムのプロトタイプをサーバーサイドからクライアント、Android アプリまで一貫して開発しました。
- サーバーサイドで Scala、Android アプリで AndroidAnnotations、Javascript の FW に AngularJS と、私にとって初めての言語・FW ばかりを選ぶことができ、とてもありがたい環境でした。
- ユーザー様との打ち合わせや仕様の検討に参加させていただき、仕様や機能、意図がくみとりやすかったので私の思いつく範囲で機能のご提案をしました。
- コンパクトな体制でしたしまだ仕様が固まっていない状態での開発でしたので、早く利用イメージができるよう、できるだけスピード感を持って開発するよう心がけました。
- テーブル数 27
- 担当フェーズは設計、開発、テストです。

2014-04-01 TO 2014-08-31

ネイティブアプリエンジニア (開発チーム 5 名) at SDK 運用チーム

Windows, Linux, Java, Gradle, C++, Cocos2dx, SQLite, Jenkins, Nexus, Git, SVN

- この時期から社内の Android アプリが認証や課金時に使用する SDK の担当になりました。
- 複数の SDK がありましたのでリリース作業自動化や品質向上のための各静的分析自動化など、jenkins を使って更に安全に運用できるよう努めました。
- Gradle でビルドを行いたいと要望がありましたので SDK のビルド・リリース作業の Gradle 化を行いました。
- また、実現したい機能が既存のプラグインだけでは実現できなかったり無理して組み込んでもパフォーマンスが悪く使い物にならなかったりしたので独自の jenkins プラグインを作成し組み込みました。
- SDK の Cocos2dx プラグイン化作業(Android 側と共通部分)を行いました。
- 担当フェーズは設計、開発、単体テストです。

2013-09-27 TO 2014-03-31

ネイティブアプリエンジニア (開発チーム 5 名) at アプリ運用チーム

Windows, Java, Shell(少し), Groovy(少し), SQLite, Git, Jenkins

- 客先のメイン事業のサービス名を冠したアプリで長く運用されており、クローズできないアプリでした。
- 歴史があり、ソースが読みにくいので数ヶ月のリファクタリング期間が設けられていました。
- 画像選択を行う画面をライブラリとして開発しました。
- クライアントサイドをメインで開発したことがなかったため、リソースの節約やイベントの多さ、ケースの複雑さ、端末依存の現象や UI のためだけの制御など、サーバーサイドとの違いに戸惑いました。
- 品質を維持するための静的解析やビルド自動化の作業の時間ももらえたため、チームのサーバーに jenkins を構築しました。
- 担当フェーズは設計、開発、単体テストです。

2012-12-06 TO 2013-09-26

エンジニア (開発チーム 14 名) at ゲーム部門コアシス

Linux, Java, FTL, HTML, CSS, JavaScript, shell, Spring, MySQL, SVN, Git, Apache, Tomcat, Memcached, Jenkins, Android

- 複数ゲームで使用されるシステムの開発、運用。
- 認証などは大元のプラットフォーム側で運用していたことと、各ゲーム側にもこちらから使う API を用意してもらって構築したため普通のシステムより多くの API を使用するシステムになりました。
- パフォーマンスの問題や、大型ゲームのトラフィックがリリースしたての小さなゲームの API サーバーを圧迫するなど、使用 API が多いシステムならではのトラブルがありました。処理の頻度を緩めて各サーバーへのアクセスを短時間に集中させないこと、キャッシュを許容するよう、数値の管理チームに相談に行くことで対応しました。
- 各ゲームとの連携がうまく行かないと動かず、立場上も反感を買いかねない場所ですのでできるだけ連絡を密にするよう気をつけ、調査などもこちらでできることは吸収するよう努めました。各ゲームでサーバーの構築方法や FW が違いましたので調査やソースの確認を通していろいろなケースの勉強になりました。
- アプリケーションの作りもアクセスの多いサイトならではの方法が多く、とても勉強になりました。
- AP サーバー 15 台程度(当時)
- DB サーバー 3 台(master1&slave2・当時)
- 画面数 20 程度
- テーブル数 50 程度
- 各ブラウザゲームの WebView アプリ化(Android)もさせて頂きました。対象ゲームは　6 個、今後さらに増える見通しでしたので計測などゲーム個々の処理以外は全て FW として実装し、各ゲームの処理は数個程度の薄いクラスで収まるよう作成しました。
- 担当フェーズは設計、開発、テストです。

2012-08-16 TO 2012-12-05

エンジニア (開発チーム 20 名) at モバイルブラウザゲーム運用

Linux, Java, FTL, HTML, CSS, JavaScript, shell, Spring, Seaser, MySQL, Apache, Tomcat

- フィーチャーフォン、スマートフォン用ブラウザゲームの運用
- アクセス数が多いのでサーバー 40 台程度で運用していました。
- アプリケーションの作りもアクセスの多いサイトならではの方法が多く、とても勉強になりました。
- 担当フェーズは追加機能開発(機能ごとの DB 設計〜単体テストまで), CS 調査です。

2012-05-01 TO 2012-08-15

エンジニア at 独自サービス構築

Linux, Java, JSP, HTML, CSS, JavaScript, shell, Struts, MySQL, Apache, Tomcat

- サイト構築システム
- 画面数 83
- テーブル数　39
- 担当フェーズは全行程です。

2011-10-07 to 2012-04

エンジニア (開発チーム 13 名) at 大手家電メーカー事業部生産管理システム構築

Windows, Java, JSP, PL/SQL, Struts ベース独自 FW, Oracle, Apache

- 大手家電メーカー事業部生産管理システム
- Ui の処理は Java, JSP で行い、DB への更新は PL/SQL で行う
- 画面数 73
- テーブル数　122
- 担当フェーズはJava 側の詳細設計、開発、テストです。
- 詳細設計から実装、テストまで
- リリース前プログラム修正、調査、チューニング等
- Java 開発者は当初私を含め 4 人アサインされており、私自身は 11 月末までの短期案件の予定でした。開発フェーズが落ち着きメンバーが減る段階では、技術力を評価していただき、チームの Java 担当技術者としては唯一、3 月末のリリース前後の要員として残ることになりました。保守要員としての長期の依頼も頂く等、評価していただきました。
- 開発スタイルは WF ですが全体的に時間と要員が足りず、設計書がないまま口頭で仕様を聞いて開発するなど、イレギュラーな状態での開発になりました。
- 設計書がなく、特に他の技術者の担当プログラムはソースしか情報が無い為、変更や修正の際にはソースからプログラム設計を理解し変更を行いました。
- Web アプリケーションについてはチームの中で私が一番詳しかった為、機能実現に向けた技術的な提案や Web 特有の制約事項の説明を行い、設計に協力しました。
- リリースも近く、スケジュールもぎりぎりだったため、いつも以上に体調管理に気をつけました。お蔭様で 1 度も病欠せず、業務に専念することが出来ました。

2008-10 to 2011-10

エンジニア at 中堅ITコンサル社内ベンチャー

Windows, Java, JSP, HTML, CSS, JavaScript, Struts2, MySQL, Apache

- 大手飲食チェーン店ポイントシステム、部品バッチ作成、選挙の後援者管理システム、ツイッター割引まとめサイト、パリーグ野球チームの分析システム機能追加、パリーグポータルサイト管理サイト、携帯サイト CMS、美容院予約システム、等構築。

2011-02-21 to 2011-10-07

エンジニア (開発チーム 4 名, デザイン会社数名) at 大手飲食チェーン店依頼ポータルサイト

Linux(RH), Java, JSP, Struts2, MySQL, Apache

- 大手飲食チェーン店より依頼の、飲食店ポータルサイト構築。
- 画面数 26
- テーブル数 85
- 基本設計から実装、テストまで
- デザインはデザイン担当会社から HTML でデザインを受け取り、JSP への変換を行う。
- JSP はデザイン会社の人が直接更新することもあるのでできるだけ HTML がわかれば理解できるソースにするよう心がけました。
- すでに構築されているモバイルサイトの派生プロジェクトですのでソースや設計はモバイルサイトに準じた方法をとりました。
- その他、別に製作してあるモバイルサイトの管理サイトに PC サイト用の機能追加を行いました。

2009-04-01 TO 2009-05-31

エンジニア (開発チーム 3 名) at 小売店向け EC サイト構築・受注発送請求管理システム構築

Linux(RedHat), Java, JSP, HTML, JavaScript, CSS, Struts2, MySQL

- 当時エクセルで行っていたグループ会社向け通販の管理を Web システム化するプロジェクト。
- 顧客サイト画面数 25
- 管理サイト画面数 48
- テーブル数 33
- 担当フェーズは要件定義。基本設計、DB 設計、詳細設計、実装、デザインあて、テスト、納品、データ移行です。
- スケジュール管理
- 顧客向け EC サイトとクライアント向け管理サイト要件定義、各設計、実装、テスト、納品、データ移行
- ヤマト運輸発送伝票出力システム B2 との連携、佐川急便発送伝票出力システム e 飛伝との連携
- 要件定義から 2 ヶ月で納品し稼動させるスピード開発でしたが移行までスケジュールどおりに進めることができました。
- 管理サイト構築後の使用感の確認を行った際に担当者が IT リテラシーが低めだとわかり、業務マニュアルとヘルプを兼ねる「ガイド」を担当業務別に作成、ログイン後のトップページやメニューは担当者が使う機能数個のみに絞り、迷うことがないよう工夫したところ、大変喜んでもらうことができました。
- 個人情報を扱うため、厳密な権限管理ができるようにしました。
- 運用負担がかからないよう、クライアント側で各種設定ができるようにしておきましたので問い合わせは 2 年半たっても数件にとどまりました。

2009-02 TO 2009-04

エンジニア (開発チーム 5 名) at 受託飲食店グループ向けポイントシステム開発

Linux(RedHat), Java, JSP, HTML, JavaScript, CSS, Struts, MySQL, Apache,

- 大手飲食店グループ向けポイント管理・CRM システム構築・運営。
- モバイルサイトへくじ機能追加
- 分析システムで使用する天気情報取得バッチ作成
- ポイント失効処理、失効前メール送信バッチ作成
- 上記機能設計、実装、テスト
- モバイルサイトの作成や実行可能 Jar の作成、メール送信機能は初めてでしたがインターネットで情報を集め自力で作成することができました。

2008-10 to 2009-01

エンジニア (開発チーム 1 名) at 顧客・会合管理システム(受託)

Linux, Java, JSP, HTML, JavaScript, CSS, Struts, MySQL, Apache

- クライアント主催の会合やワークショップに参加している顧客情報を管理し、アテンド担当社員情報と会合の連絡管理や出欠管理を行うシステムの構築。
- 画面数 46
- テーブル数 22
- 要件定義、基本設計、DB 設計、画面設計、実装、デザインあて、テスト、運用保守、データ移行サポート
- スケジュール管理、進捗管理(デザイナー、HTML コーディング担当との調整)
- 最初の 1 ヶ月はクライアント側の調整が終わっていなかったことと、私自身 Java での開発が初めてだったため練習用サンプルプログラムを作成する期間となり、実質 2 ヶ月間で要件定義から納品まで行うスピード開発でした。
- 初めてのプロジェクト全体スケジュール管理を行いましたが約束の納品日を守ることができました。
- クライアントは社内の人事異動や部署名変更が頻繁で、顧客アテンド担当者の管理が大変だったとの事なのでアテンド担当者の所属部署やアテンド担当者をすべて別の担当者に引き継ぐなど、一括操作ができるよう利便性に考慮しました。
- 個人情報を扱うため、機能画面ごとの厳密な権限管理をし、権限は役割に合わせて一括で管理、ユーザー側で自由に変更できるよう設計を

2008-03 to 2008-09

エンジニア (開発チーム 1 名) at ガラケー用お小遣い帳 Web アプリ, ガラケー用支出記録・割勘金額管理用のモバイルサイト (Web 勉強用)

Windows, PHP, MySQL

- SAP プロジェクトからの帰宅後や休日を使用して PHP で構築しました。
- お小遣い帳 Web アプリはクレジットカード、電子マネー、現金を一括で管理するサイトが欲しかった為、構築しました。
- 支出記録・割勘用モバイルサイトは複数人の共通の支出を記録し、全員が平等の金額を負担する為、携帯サイトとして構築しました。
- 主にみんなで外出した際、代表者が支払う事が多かったので最終的に誰がどれだけ払っているのか分からなくなるため、使用しました。
- 担当フェーズは全行程です。

2008-06 to 2008-09

エンジニア (開発チーム 10 名程度, プロジェクト規模百数十名程度) at 大手化学メーカー SAP アドオン開発

SAP, ABAP, Oracle, Windows

- SAP 導入・アドオン開発。
- 新規プログラム詳細設計、詳細設計書作成、DFD 作成、実装、テスト仕様書作成、テスト
- 仕様変更対応
- パフォーマンスチューニング
- 調査
- 通常の新規モジュール開発の他に新人 4 名での画面プログラム開発、複数回にわたり開発停止になった画面プログラムを外国籍のメンバー数名と開発するなど、大きなプログラムを開発することもできるようになりました。
- 移行対象モジュールが使用しているモジュールを正確に知ることができずに設計チームが困っていることを知り、空き時間で同時移行するべきモジュールを一覧にするプログラムを作り、提供しました。
- 二次受けの企業と共同で行う ABAP 講習の講師を 1 日のみ依頼され汎用モジュール開発の講習を行いました。
- 担当フェーズは詳細設計、開発、単体テストです。

2007-08 TO 2008-05

エンジニア (開発チーム 10 名程度, プロジェクト規模百数十名程度) at 大手家電量販店 SAP アドオン開発

SAP, ABAP, Oracle, Windows

- SAP 導入・アドオン開発。
- 新規プログラム詳細設計、詳細設計書作成、実装、テスト仕様書作成、単体テスト
- 仕様変更対応
- 調査
- OJT で先輩に助けられて経験を積みました。
- プログラムを始めて半年の頃に作った個人用の調査プログラムが好評で、設計チームでも使われることになり、割り当てられた仕事以外でも貢献することができました。また、このツールなしでは仕事ができないと今でもプロジェクトに残っている先輩に言っていただき、今でも使われていることを知りました。作ったものが便利だと思ってもらえることを非常に嬉しく思います。
- チームにはプロジェクトの初期メンバーとしてアサインされましたが、当初 6 名程度だったチームは評判が良く、要員追加されて十数名程度にまでなりました。開発フェーズが終盤になると当時の 1 次受けコンサルティング会社の別プロジェクトへチームまるごと移動するよう依頼される等チームの評価向上に微力ながら貢献しました。
- 担当フェーズは詳細設計、開発、単体テストです。

自己PR TO

クライアントの作業効率改善に向けた提案:

- 業務系の開発は顧客が気づかない作業効率改善箇所の指摘と改善の為の機能追加や、操作に迷わない画面設計と機能を追加する工夫をしてクライアントに喜ばれました。
- 例えば、配送担当者は主に 6 つの業務を行っているので、配送担当者が管理画面にログインするとその 6 つの業務のうちどれを行いたいかの選択肢のみを表示し、
- 発送伝票作成であれば、「○ 月 ○ 日の発送伝票を作成する」というボタンを表示することで迷わないよう設計しています。
- また、納期までの期間が短く、運用契約のされているプロジェクトでも各種設定画面と、それぞれの画面上に詳細な説明文を用意し、変更や操作がわからないなどの問い合わせを
- 減らす工夫をしています。運用開始から 2 年程度たちましたが問い合わせは2 〜 3 件にとどまっています。

TO

開発効率化に向けた取り組み:

- 文字列変換や入力チェック用部品、年齢や期間の算出など、プロジェクト特有でない処理は汎用部品として独立させ、次のプロジェクトでは部品を組み合わせるだけでできるよう、汎用部品を日々洗練させています。すでにテスト済みの部品なので、プロジェクト全体のバグの数を減らし、開発スピードを上げることができるようにしています。
- また、画面設計の変更やデータベースの変更に強い設計を目指して構築していくうちに、画面やデータを扱う箇所を分離して構築するようになっていましたが、後に MVC と呼ばれることを知りました。MVC に関してはすでに知られている設計手法でしたが常に開発効率と汎用性、堅牢性を意識して開発しています。
- また、各ツールの開発や Jenkins でのビルドやテスト、静的解析はもちろんのこと、ADT の更新通知やリリース時のドキュメント収集保存・apk 調査用 job 等、自動化で各所でのエンジニアのストレスを軽減しシステムの品質向上・開発スピードアップできるよう常に意識しています。

TO

個人での勉強:

- 2 社目の会社に入社する時、クライアントサーバーモデルや HTTP について一切分からない状態でしたが独学で PHP のシステムをつくり、認められて入社しました。
- Web 業界に入った後、占有サーバーを借りて Xen で仮想化し、それぞれに Apache や DB などアプリケーションをインストールして AP サーバー・DB サーバーにし、個人で作ったサイトをそちらで運用しています。GNOME をインストールしたサーバーには VNC で接続して普段使うデスクトップサーバーとして構築、普段から仮想デスクトップ環境で開発やメール、スケジュール確認など PC で行う自分の作業をほぼ全てサーバーにて行っています。
- しばらく Xen にて運用していましたが現在は新しく Ansible, VirtualBox, Vagrant を使用して構築しなおし、デスクトップサーバー、AP,DB サーバーに加え個人用の Jenkins を インストールして使用できるようにしています。
- ローカルのマシンとデスクトップサーバーは毎月 OS クリーンを行い、案件ごとの開発環境の構築を Ansible で行っていましたが各ツールのインストールや設定に非常に時間がかかるため案件ごとにデスクトップ環境を Docker で作成し Docker 上での開発に移行しました。
- 常に OS クリーンをすることとステートの位置を把握することでローカルのみに重要なデータを保管してしまったり、ローカルだけで動くなどの問題が発生しないよう努めています。
- また、常に自分のローカル環境とデスクトップサーバーを作り直すことで新しく join したメンバー向けに配布している開発環境構築のスクリプト(最近はほとんど Ansible の playbook です)が正しく動作するかを検証しています。
- また、最近は趣味でも仕事でも電子工作をはじめ、 arduino を使用して 赤外線、各種センサー、サーボモーター等を使ったり、CAD でデータを作って 3D プリンタで出力できるようになりました。今は電気の回路設計を勉強しているのと、レーザーカッター、切削等の工作機械、塗装等のトレーニングも受け、自分の作りたいものの技術的な課題を消化しています。
- 今後、業務や個人の趣味で困難なことがあっても自力で調べて対処できると思っています。
- よろしくお願い致します。

EDUCATION

2004-04-01 TO 2007-03-31

Collage

- *Qualified Bornsetter*

LANGUAGES

日本語	ネイティブ
英語	ビジネス会話

PROFILES

GitHub Meeting (30min) LinkedIn
Twitter

PERSONAL

Home Automation, IoT, Zapier, IFTTT, Universal Remote Control
Mirror Polish, Film, Buffing Compounds